Section 05: Prebuilt ReAct Agent

The Reason + Act Pattern as a Graph

LangGraph v1.0.x (March 2026)

Mental Model: ReAct Pattern

The Big Picture

- **ReAct**: Reason + Act (Yao et al., 2023)
- Agent **thinks** about a problem, **acts** via tools, **observes** results
- Loop: Think -¿ Act → Observe → Think → ...
- Much more effective than just chain-of-thought

Why It Works

Grounding reasoning in real-world interaction beats pure reasoning.

What Is ReAct: The Loop

ReAct Cycle

- **Observe**: Read the problem and current context
- **Think**: Reason about what to do next
- **Act**: Call a tool to gather information or take action
- **Observe**: See the result, decide to continue or stop
- **Repeat** until goal is reached or max steps exceeded

Key Advantage

Can recover from mistakes: if a tool call fails, agent **reasons** about the failure.

create_react_agent: The High-Level Shortcut

Basic Usage

```
from langgraph.prebuilt import create_react_agent
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-4")
tools = [get_weather, search_docs, add_numbers]
agent = create_react_agent(model, tools)
result = agent.invoke({
  "messages": [HumanMessage("What's the weather?")]
})
print(result["messages"][-1].content)
```

One-Liner Magic

`create_react_agent()` builds a complete agent in seconds.

What create_react_agent Builds Under the Hood

Equivalent Manual Graph

```
builder = StateGraph(MessagesState)
builder.add_node("agent", lambda state: {
  "messages": [model.invoke(state["messages"])]
})
builder.add_node("tools", ToolNode(tools))
builder.add_edge(START, "agent")
builder.add_conditional_edges(
  "agent", tools_condition,
  {"tools": "tools", END: END}
)
builder.add_edge("tools", "agent")
graph = builder.compile()
```

Demystified

`create_react_agent` is just StateGraph + ToolNode + tools_condition.

TikZ Diagram: ReAct Agent Graph Structure

Agent Loop

Tools flow back to Agent. Loop continues until agent stops calling tools.

Customizing the System Prompt

Custom Prompt Example

```
from langchain_core.prompts import ChatPromptTemplate
system_prompt = """You are a helpful AI assistant.
Always be concise. Use tools when needed."""
prompt = ChatPromptTemplate.from_messages([
  ("system", system_prompt),
  ("placeholder", "{messages}")
])
agent = create_react_agent(
  model,
  tools,
  prompt=prompt
)
```

Prompt Matters

System prompt shapes agent behavior. Experiment with tone, constraints, guidelines.

Adding a Checkpointer for Persistence

Persistence Setup

```
from langgraph.checkpoint.memory import MemorySaver
checkpointer = MemorySaver()
agent = create_react_agent(
  model,
  tools,
  checkpointer=checkpointer
)
config = {"configurable": {"thread_id": "user_123"}}
result = agent.invoke(
  {"messages": [HumanMessage("Hi")]},
  config=config
)
```

Enables Resumption

Checkpointer + thread_id allow conversations to persist across invocations.

Adding Message History and Memory

History Pattern

```python
def load_messages_from_db(user_id):
  """Simulate loading from database."""
  return get_user_history(user_id)
config = {"configurable": {"thread_id": "user_123"}}
old_messages = agent.get_state(config)["messages"]
new_input = HumanMessage("Follow up question?")
result = agent.invoke(
  {"messages": old_messages + [new_input]},
  config=config
)
```

Stateful Agents

Checkpointer handles message history automatically if you `invoke` with config.

Customizing with state_modifier

state_modifier Hook

```python
def modify_state(state):
  """Pre-process state before model invocation."""
  # Add context, filter messages, etc.
  if len(state["messages"]) > 10:
    state["messages"] = state["messages"][-10:]
  return state
agent = create_react_agent(
  model,
  tools,
  state_modifier=modify_state
)
```

Advanced Customization

state_modifier lets you inject custom logic before model invocation.

Pre-processing and Post-processing Hooks

Custom Hooks

```
def log_invocation(state):
  print(f"Messages: {len(state['messages'])}")
  return state
def log_completion(state):
  print(f"Final message: {state['messages'][-1]}")
  return state
agent = create_react_agent(model, tools)
agent = agent.map_nodes(
  lambda node: (log_invocation >> node >> log_completion)
  if node.name == "agent" else node
)
```

Debugging and Observability

Hooks enable logging, monitoring, and state inspection.

When to Use Prebuilt vs Custom Graph

Use Prebuilt ReAct If

- Simple reason-act loop is enough
- Standard tool calling, no custom nodes
- Want fast prototyping

Use Custom StateGraph If

- Multiple agent teams or parallel branches
- Complex approval/human-in-the-loop workflows
- Custom node logic, state transformations

Comparison: Prebuilt vs Custom

Aspect	Prebuilt	Custom
Lines of code	5–10	50–200
Setup time	Seconds	Minutes
Flexibility	Limited	Full
Debugging	Opaque	Transparent
Multiple agents	No	Yes
Human interrupts	Limited	Full

Strategy

Start with prebuilt. Migrate to custom when you hit limitations.

Limitations of the Prebuilt Agent

What You Can't Do

- No parallel tool execution
- No inter-agent communication
- Limited human-in-the-loop control
- Can't easily add custom logic between nodes
- Fixed agent -¿ tools $\rightarrow$ agent loop

When to Outgrow

Once you need more than reason + act, build a custom graph.

Migrating from Prebuilt to Custom

Refactor Pattern

```
# Was using prebuilt
agent = create_react_agent(model, tools)
# Now building custom
from langgraph.graph import StateGraph, START, END
builder = StateGraph(MessagesState)
builder.add_node("agent", agent_node)
builder.add_node("tools", ToolNode(tools))
builder.add_edge(START, "agent")
builder.add_conditional_edges(
  "agent", tools_condition, ...
)
custom_agent = builder.compile(...)
```

Smooth Transition

Extract prebuilt logic and customize node by node.

Summary: ReAct Cheat Sheet

Quick Start

```python
from langgraph.prebuilt import create_react_agent
agent = create_react_agent(
  ChatOpenAI(model="gpt-4"),
  [tool1, tool2, ...]
)
result = agent.invoke({
  "messages": [HumanMessage("Task")]
})
print(result["messages"][-1])
```

Remember

Prebuilt ReAct is fast, but custom graphs are powerful. Pick the right tool.

Self-Check: Do You Understand?

1. What does ReAct stand for and why is it effective?
2. How many lines of code are in a minimal `create_react_agent` call?
3. What does `state_modifier` let you do?
4. When should you build a custom graph instead of using prebuilt?
5. How does persistence (checkpointer) change agent behavior?
6. What's the difference between Prebuilt and StateGraph approaches?
7. How do you add custom logic to a prebuilt agent?